Isiliel v1

Programming Manual/Tutorial

Embedded scripting language with a stack VM,
classes, tables, coroutines, graphics, and a REPL

Eben (AI-assisted)

Version 1.0

November 11, 2026

This document describes the language as implemented in version 1.0.
Whenever the project documentation and the code diverge, the code is the source of truth.

Contents

1 About This Manual **2**

1.1 How to Read . . . 2

1.2 Conventions . . . 2

1.3 Note on Known Differences . . . 2

I Getting Started **3**

2 Overview **4**

2.1 Main Features . . . 4

2.2 Execution Model . . . 4

2.3 Target Audience and Portability . . . 5

3 Installation and First Steps **6**

3.1 Building . . . 6

3.2 Running a Script . . . 6

3.3 The REPL . . . 6

3.4 First Complete Program . . . 7

3.5 Errors and Traceback . . . 7

4 Lexical Structure **8**

4.1 Comments and Statements . . . 8

4.2 Identifiers and Keywords . . . 8

4.3 Numeric Literals . . . 8

4.4 Strings . . . 8

4.5 Operators . . . 9

II The Language **10**

5 Values, Types, and Expressions **11**

5.1 Available Types . . . 11

5.2 Integers and Floating Point . . . 11

5.3 Strings . . . 11

5.4 Truth and Falsity . . . 12

5.5 Logical Operators . . . 12
5.6 Precedence . . . 12
5.7 Assignment and Conversions . . . 13

6 Variables and Scope 14
6.1 Globals and Implicit Locals . . . 14
6.2 Optional Scalar Declarations . . . 14
6.3 Multiple Assignment . . . 15
6.4 Recursion and Local Cleanup . . . 15
6.5 REPL . . . 15

7 Control Flow 16
7.1 if / elseif / else . . . 16
7.2 while . . . 16
7.3 Classic for . . . 16
7.4 repeat / until . . . 16
7.5 break, continue, and goto . . . 17
7.6 Tail Calls . . . 17

8 Functions 18
8.1 Definition and Call . . . 18
8.2 Multiple Returns . . . 18
8.3 Varargs . . . 18
8.4 Functions as Values . . . 19
8.5 Anonymous Closures . . . 19
8.6 global inside Functions . . . 19
8.7 pcall and Traceback . . . 20
8.8 Builtins as Functions . . . 20

9 Strings 21
9.1 Basic Operations . . . 21
9.2 Slices . . . 21
9.3 Comparison and Concatenation . . . 21
9.4 Conversion and Formatting . . . 21
9.5 Arduino String API . . . 22
9.6 Patterns . . . 22
9.7 UTF-8 . . . 23

10 Lists and Matrices **24**

10.1 1D Lists . . . 24
10.2 Slices and Copies . . . 24
10.3 2D Matrices . . . 24

11 Tables **25**

11.1 Creating and Accessing . . . 25
11.2 Equality and Reference . . . 25
11.3 Iteration . . . 25
11.4 Textual Representation . . . 26
11.5 Metatables . . . 26

12 Structs and Classes **27**

12.1 Structs . . . 27
12.2 Classes and Inheritance . . . 27
12.3 Reference and Equality . . . 28
12.4 Structs versus Tables . . . 28

13 Metatables and Garbage Collection **29**

13.1 setmetatable and getmetatable . . . 29
13.2 Supported Metamethods . . . 29
13.3 Cycle Collection . . . 29
13.4 Limitations . . . 30

14 Modules and Dynamic Code **31**

14.1 include . . . 31
14.2 .xlb Libraries . . . 31
14.3 require . . . 31
14.4 load and dofile . . . 31
14.5 Native .mod Modules . . . 32

15 Error Handling **33**

15.1 Compilation and Runtime Errors . . . 33
15.2 Traceback . . . 33
15.3 No try/catch . . . 33
15.4 Sandbox . . . 33

16 Threads and Coroutines **34**

16.1 System Threads . . . 34

16.2 Cooperative Coroutines 34

17 Standard Library **36**

17.1 Console and Conversion 36
17.2 Strings, Lists, and Tables 36
17.3 Mathematics and Complex Numbers 36
17.4 Time 37
17.5 Files 37
17.6 Sound 38

18 Graphics **39**

18.1 setup and draw 39
18.2 Drawing 39
18.3 Input and Images 40

19 Embedding the Interpreter in C++ **41**

19.1 Options 41
19.2 API Functions 41
19.3 Builtin Signature 42

20 Tools and Debugging **43**

20.1 Disassembler 43
20.2 Tests and Sanitizers 43
20.3 Project Structure 43
20.4 Debugging in VS Code 43

21 Known Limitations and Differences **44**

21.1 Language 44
21.2 Metatables and Library 44
21.3 Concurrency and Memory 44
21.4 Roadmap 45

A Grammar Summary **46**

B Quick Builtin Reference **47**

C Complete Programs **50**

C.1 FizzBuzz 50
C.2 Iterative and Recursive Fibonacci 50
C.3 Counter Closure 50

C.4 Table and Iteration 51
C.5 Class with Inheritance 51
C.6 Metatable for Addition 51
C.7 Coroutine 52
C.8 Simple Graphics 52

D Glossary **53**

Chapter 1

About This Manual

This is a practical manual and an introductory reference for the Isiliel language. It is written for people who want to *use* the language: write scripts, organize code into functions and modules, use tables and classes, draw on screen, control coroutines, and embed the interpreter in a C++ program.

The text assumes that you know how to program in some imperative language (C, Python, Lua, or JavaScript). You do not need to know the internal implementation, but some chapters mention behavioral details to help you avoid pitfalls.

1.1 How to Read

- Chapters 3 through 8 form the basic tutorial: installation, values, variables, control flow, and functions.
- Chapters 9 through 13 cover the data types and object-oriented programming.
- Chapters 14 through 19 cover modules, errors, concurrency, the standard library, graphics, and embedding.
- The appendices contain the grammar summary, a quick builtin reference, and complete programs.

1.2 Conventions

- Gray-background blocks are Isiliel code. Comments use `//`, and example text avoids accents inside code so it remains compatible with any editor.
- `println`, `printf`, and function names appear in a monospaced font.
- Paths such as `scripts/script_1.isi` are relative to the project root.
- "v1.0" refers to the version described in this manual.

1.3 Note on Known Differences

The language is evolving. Some points in the project documentation (`docs/roadmap.md` and `docs/linguagem.md`) describe the *planned* behavior; this manual describes the *current* behavior. The most important differences are collected in Chapter 21. In particular, in v1.0:

- `true` and `false` are numeric literals (`1` and `0`); there is no distinct runtime boolean type.
- `&&` and `||` evaluate both operands and produce `1`/`0`; there is no short-circuiting.
- `!` is bitwise negation, not logical negation.
- `pairs`/`ipairs` return lists; there is no `for k,v in ...`.

Part I

Getting Started

Chapter 2

Overview

Isiliel is a dynamic scripting language implemented in C++17 with the pipeline:

source → lexer → bytecode compiler → stack VM

The same executable provides a file *runner*, an interactive REPL, a disassembler, and an embedding API. The language was designed to be small and portable, with a focus on embedded systems, but it includes features found in general-purpose scripting languages.

2.1 Main Features

- Exact 64-bit integers and double-precision floating-point numbers; complex numbers with the `j` suffix.
- Immutable strings with escapes, indexing, slicing, and patterns.
- Arduino String API (free functions) and C99 `math.h` functions/constants.
- 0-based lists/matrices with slicing, `append`, `sort`, and basic linear algebra.
- Tables (dictionaries) with text, numeric, or boolean keys.
- First-class functions, multiple returns, varargs, and anonymous closures.
- Structs and classes with single inheritance, an `init` constructor, and `super`.
- Metatables and metamethods (`__index`, `__call`, `__add`, `__eq`, `__tostring`, `__gc`).
- Cycle collection by *trial deletion*.
- Modules through `include`, `require`, `load`, and `dofile`; native `.mod` modules.
- System threads and cooperative coroutines.
- Processing-style 2D graphics with a headless software backend or raylib.
- REPL, automated tests, ASan/UBSan, fuzzer, and benchmarks.

2.2 Execution Model

A script is compiled entirely to bytecode before execution. The VM is stack-based and keeps global and local symbols in a symbol table. The compiler resolves names at compile time when possible; direct calls use the function address, while function values use indirect calls.

Running a file prints diagnostic banners and returns exit code `0` on success or `1` on error. The REPL suppresses the banners and keeps symbols and bytecode between lines.

2.3 Target Audience and Portability

The main implementation targets desktop systems, but the code avoids mandatory operating-system dependencies. The graphics backend is abstract (`software` or `raylib`), and the `make embedded` profile uses `-Os` without raylib. Threads and files are optional in embedded environments; see `docs/proposta_embarcada.md` for the design decisions.

Chapter 3

Installation and First Steps

3.1 Building

From the project root:

```
make                # build/isirun in debug mode (-O0 -g3)
make test           # full test suite (164 cases in v1.0)
make release        # -O2 -DNDEBUG -flto
make embedded       # -Os, no raylib (embedded profile)
make asan           # suite with AddressSanitizer + UBSan
make fuzz           # deterministic fuzzer
make bench          # benchmark against Lua 5.4
```

The `raylib` backend is detected through `pkg-config`; use `make USE_RAYLIB=1` to force it or `make USE_RAYLIB=0` to disable it. Without raylib, everything runs on the `software` backend (headless, writing `.ppm` images).

3.2 Running a Script

```
./build/isirun scripts/script_1_matriz_lu.isi
./build/isirun --disasm scripts/script_1_matriz_lu.isi   # shows the bytecode
./build/isirun                                  # interactive REPL
```

A script is a text file (usually `.isi`) with statements terminated by `;`. First example:

```
// hello.isi
nome = "mundo";
println("Ola, " + nome + "!");
```

Output:

```
Ola, mundo!
```

3.3 The REPL

With no arguments, the executable starts a REPL. State (variables, functions, and bytecode) persists between lines. Blocks are entered over multiple lines; the `...` prompt continues until the braces are closed. A simple statement ends with `;`; a block ends with `}`. `exit`, `quit`, or EOF terminate the session.

```
>>> a = 2 + 3;
>>> println(a);
5
>>> f(x) { return x * x; }
```

```
>>> println(f(7));
49
```

Compilation or execution errors are reported but do not end the session. The REPL enforces growth limits to prevent unlimited memory consumption (default: 250000 instructions and 50000 symbols). Adjust them with the environment variables `MYINTERP_REPL_CODE_LIMIT` and `MYINTERP_REPL_SYMBOLS_LIMIT`; `0` removes the limit.

3.4 First Complete Program

```
// media.isi: reads two numbers and computes the average
a = 7;
b = 9;
media = (a + b) / 2;
printf("media = %g\n", media);

if (media >= 7) {
    println("aprovado");
} else {
    println("reprovado");
}
```

Note that `/` always produces a floating-point number. For integer division use `floordiv(a, b)`.

3.5 Errors and Traceback

Compilation errors include the line:

```
ERRO COMPILACAO: Esperado ';' apos a expressao (linha 4)
```

Execution errors include the line and, when available, a traceback:

```
ERRO EM TEMPO DE EXECUCAO: Divisao por zero (linha 7)
  em quebra (linha 9)
```

Use `pcall` to catch runtime errors (Section 8.7).

Chapter 4

Lexical Structure

4.1 Comments and Statements

Comments start with `//` and run to the end of the line. There are no block comments. Simple statements end with `;`. Some statements (function definitions, `include`, `import`, `start`/`stop`/`pause`/`resume`) accept an optional semicolon; use `;` for consistency.

4.2 Identifiers and Keywords

Identifiers start with a letter or `_` and may contain letters, digits, and `_`. The reserved words are:

`if`	`else`	`elseif`	`while`
`for`	`return`	`break`	`continue`
`void`	`int`	`float`	`fn`
`class`	`extends`	`new`	`super`
`include`	`import`	`type`	`global`
`thread`	`start`	`stop`	`pause`
`resume`	`repeat`	`until`	`goto`
`nil`			

`true`, `false`, `i32`, `f64`, `bool`, and `str` are not keywords: they are literals or prefixes recognized by the parser. That is why `type(x)` (the function) and `type Nome {}` (the struct definition) coexist without conflict.

4.3 Numeric Literals

- Decimal integers: `0`, `42`, `9223372036854775807`. Literals that fit in `int64` are exact.
- Floating point: `3.14`, `0.5`. The lexer does not accept scientific notation: `1e9` is read as `1` followed by the identifier `e9`.
- Complex numbers: suffix `j` or `J`, as in Python: `3j`, `1+2j`, `2.5J`.

4.4 Strings

Strings are delimited by double quotes. Accepted escapes: `\n`, `\t`, `\r`, `\\`, `\"`, and `\0`. Unknown escapes keep the backslash (Python style): `"\q"` becomes `\q`. Strings are immutable.

```
s = "linha 1\nlinha 2\tfim";
println(len(s));      // number of characters
println(s[0]);        // first character
println(s[-1]);       // last character
```

4.5 Operators

Category	Operators
Arithmetic	`+ - * / %`
Bitwise	`& | ^ ~« »`
Comparison	`== != < > <= >=`
Logical	`&& ||`
Assignment	`= += -= *= /=`
Access	`. [] ()`

There is no `and`, `or`, `not`, `#`, `..`, or `//` operator. `//` is a comment; for integer division use `floordiv(a,b)`. For string concatenation use `+`. For logical negation use an explicit comparison, such as `x == 0` or `x == nil`.

Part II

The Language

Chapter 5

Values, Types, and Expressions

5.1 Available Types

The table below shows the values the programmer sees and what `type(x)` returns.

Type	Example	`type(x)`
Integer	`30`, `-7`, `0x1F`	`number`
Floating point	`3.14`, `7 / 2`	`number`
String	`"abc"`	`string`
Boolean	`true`, `false` (1/0 in v1.0)	`number`
List/matrix	`[1, 2, 3]`, `[[1,2],[3,4]]`	`array`
Complex	`3j`, `1+2j`	`complex`
Struct/class	`Ponto{1, 2}`	`struct`
Table	`{nome = "Ana"}`	`table`
Function	`fn(x) { return x; }`	`function`
Coroutine	`cocreate(f)`	`thread`
Absent	`nil`	`nil`

5.2 Integers and Floating Point

Integers that fit in `int64` are exact. `+`, `-`, and `*` between integers wrap around at 64 bits. `/` always returns a `double`:

```
println(7 / 2);       // 3.5
println(7 % 2);       // 1
println(floordiv(7, 2)); // 3
println(9223372036854775807 + 1); // -9223372036854775808
```

Mixing an integer and a `double` promotes to `double`. Bitwise operators require integers; `5.5 & 1` is an error.

5.3 Strings

Strings are immutable. The `+` operator concatenates two strings; mixing a string and a number is an error (there is no silent coercion). Use `str(x)` or `printf` to convert.

```
nome = "Ana";
println("Ola, " + nome);     // Hello, Ana
println("abc" == "abc");     // 1
println("abc" < "abd");      // 1 (lexicographic)
```

5.4 Truth and Falsity

The table below describes the current implementation.

Value	Truthy?
`nil`	no
`0`, `0.0`, `0j`	no
`false` (which is `0`)	no
Any string, including `""`	yes
List, table, struct, function, coroutine	yes
Nonzero number	yes

The project documentation lists the empty string as false, but the current code treats every string as true. Use `len(s) == 0` if you want to test for an empty string.

5.5 Logical Operators

In v1.0, `&&` and `||` evaluate both operands and return `1` or `0`. There is no short-circuiting. This means that `f() && g()` always calls both functions. For conditional side effects, use `if`:

```
if (a != 0) { g(); }
```

`!` and `~` are the same bitwise negation operator. Do not use `!x` as logical negation: `!0` is `-1`, which is true. To negate a condition, write `x == 0` or `x == nil`; `if (!(x))` is not recommended.

5.6 Precedence

From lowest to highest precedence:

Level	Operators
1	assignment (`= += -= *= /=`)
2	`||`
3	`&&`
4	`|`
5	`^`
6	`&`
7	`== !=`
8	`< > <= >=`
9	`« »`
10	`+ -`
11	`* / %`
12	unary `-`, `!`, `~`
13	postfix `() [] .`

Use parentheses whenever in doubt.

5.7 Assignment and Conversions

Assignment can be used as an expression. The existing compound operators are `+=`, `-=`, `*=`, and `/=`; there is no `%=`. Useful conversions:

```
println(tonumber("42"));      // 42
println(tonumber("3.5"));     // 3.5
println(tonumber("ff", 16));  // 255
println(str(42));             // "42"
println(tostring(nil));       // nil
println(type([1,2]));         // array
```

An invalid `tonumber` returns `nil`. `str`/`tostring` produce the textual representation; `println` of a string does not add quotes, and neither does `str` of a string.

Chapter 6

Variables and Scope

6.1 Globals and Implicit Locals

At the top level of a script, an assignment creates or changes a global variable:

```
x = 10;
println(x);       // 10
```

Inside a function, an assignment to a name that is not a parameter creates a *local* variable of that function. Reads look for the local first; if it does not exist, they look for the global. To write to a global from inside a function, declare it with `global`:

```
global contador;
contador = 0;

incrementa() {
    contador += 1;
}

incrementa();
println(contador);   // 1
```

Without `global`, `contador += 1` would create a new local on each call. This is essential in the `setup()` and `draw()` callbacks.

6.2 Optional Scalar Declarations

The language is dynamic by default. Optionally, you can annotate a variable's scalar type in C/Rust style. The annotation applies coercion on every write:

```
i32 a = 42;
u8 porto = 255;
f32 p = 3.14;
f64 q = 3.14;
bool ligado = 1;
str nome = "Ana";
int i = 7;          // alias of i32
float f = 2.5;      // alias of f64
String s = "abc";   // alias of str
```

Type	Coercion rule
`i8..i64`, `isize`	truncates toward zero and checks range
`u8..u64`, `usize`	truncates and requires non-negative
`f32`	rounds to float32 precision
`f64`	normal double
`bool`	nonzero becomes 1; zero becomes 0
`str`	requires a string

The type prefix on functions, parameters, and fields is accepted and *discarded*: the language remains dynamic at those points. Only scalar variables declared with a type have automatic coercion.

6.3 Multiple Assignment

You can assign several values at once. All values are evaluated before any write:

```
a = 0; b = 0;
a, b = 3, 4;        // a=3, b=4
a, b = b, a;        // true swap
p, q, r = 1, 2, 3;
```

The number of targets must equal the number of values, except when a single call provides several values:

```
fn par() { return 10, 20; }
x, y = par();        // x=10, y=20
```

Targets are simple variables; do not use `a.b, c = ...`.

6.4 Recursion and Local Cleanup

Each function call saves and clears its locals, allowing recursion and preventing state from leaking between calls:

```
fatorial(n) {
    if (n <= 1) { return 1; }
    return n * fatorial(n - 1);
}
println(fatorial(10));   // 3628800
```

6.5 REPL

In the REPL, globals and functions persist between lines. A function's locals do not leak to the top level. Redefining a function or variable is allowed; for `type`, redefinition clears the type's fields and methods.

Chapter 7

Control Flow

7.1 if / elseif / else

```
n = 7;
if (n % 2 == 0) {
    println("par");
} elseif (n % 3 == 0) {
    println("multiplo de 3");
} else {
    println("outro");
}
```

`elseif` is a single word; `else if` also works. Conditions require parentheses.

7.2 while

```
i = 0;
while (i < 3) {
    println(i);
    i += 1;
}
```

7.3 Classic for

The only `for` in v1.0 is the C-style loop:

```
for (i = 0; i < 5; i += 1) {
    println(i);
}
```

All three clauses are optional: `for (;;) { ... }` is an infinite loop. There is no `for k,v in pairs(t)`; to iterate over tables, use `pairs(t)` and a `for` over the resulting list (Chapter 11).

7.4 repeat / until

The body executes at least once; the condition is tested at the end:

```
i = 3;
repeat {
    println(i);
    i -= 1;
} until (i <= 0);
```

`until` accepts the condition with or without parentheses.

7.5 break, continue, and goto

`break` exits the innermost loop. `continue` goes to the next iteration; in a `for` loop, the increment runs before the condition.

```
for (i = 0; i < 10; i += 1) {
    if (i == 3) { continue; }
    if (i == 7) { break; }
    println(i);
}
```

Lua-style `goto` and labels work within the same function:

```
i = 0;
::inicio::
i += 1;
if (i < 3) { goto inicio; }
println(i);   // 3
```

The label must exist in the same function; duplicates are a compilation error.

7.6 Tail Calls

`return f(args);` with a direct call to a script function is a *proper tail call*: the current frame is reused. This allows constant-space tail recursion. Indirect calls and builtins do not use this optimization.

Chapter 8

Functions

8.1 Definition and Call

A function can be defined with or without the `fn` prefix. Type prefixes (`void`, `int`, `i32`, `float`, etc.) are accepted and ignored.

```
soma(a, b) {
    return a + b;
}

fn dobro(x) { return x * 2; }
int triplo(int x) { return x * 3; }

println(soma(2, 3));   // 5
println(dobro(4));     // 8
println(triplo(3));    // 9
```

Functions are global. Parameters and variables assigned in the body are local. A function without `return` returns zero values; in a single-value context, the result is `nil`.

8.2 Multiple Returns

```
divmod(a, b) {
    return a / b, a % b;
}

q, r = divmod(7, 2);
println(q);   // 3.5
println(r);   // 1
```

In a single-value context, only the first return value is used. In multiple assignment, values are distributed in order; missing values become `nil`, and extras are discarded. `return f();` forwards all values from the callee (tail call).

8.3 Varargs

The last parameter can be `...`. Inside the function, `...` represents the extra arguments.

```
soma_var(first, ...) {
    total = first;
    for (i = 0; i < select("#", ...); i += 1) {
        total += select(i + 1, ...);
    }
    return total;
}
```

```
println(soma_var(1, 2, 3, 4));   // 10
```

`select("#", ...)` returns the number of extras; `select(n, ...)` returns from the n-th onward (1-based; negative counts from the end). `unpack(lista[, i[, j]])` returns the elements (0-based indices). `...` can only be the last argument of a call or of a `return`; in that position it expands to all values. Otherwise, `...` in an ordinary expression is the first extra value.

8.4 Functions as Values

```
dobro(n) { return n * 2; }
f = dobro;
println(f(4));        // 8
println(f == dobro);  // 1 (same function)
println(f);           // <function dobro>
```

Calling a variable that holds a function uses an indirect call. Functions cannot be used in arithmetic or converted to scalar types.

8.5 Anonymous Closures

An anonymous function `fn(...) { ... }` captures local variables from the enclosing scope by reference:

```
contador() {
    n = 0;
    return fn() {
        n += 1;
        return n;
    };
}

c1 = contador();
c2 = contador();
println(c1());   // 1
println(c1());   // 2
println(c2());   // 1
```

Two closures created in the same frame share the same upvalue.

v1.0 limitations. Only anonymous functions capture. Named functions and methods do not capture outer locals; use an inner anonymous function. Capture is lexical but not forward-looking: declare and assign the local before creating the closure. There is no IIFE: `fn(x){...}(1)` is not supported; assign it to a variable first. Scalar coercion of a captured local is not reapplied on reads/writes through the upvalue.

8.6 global inside Functions

As seen in Chapter 6, use `global nome;` to share state between functions and callbacks. Without it, assignment creates a local.

8.7 pcall and Traceback

`pcall(f, ...)` runs `f(...)` and catches runtime errors. It returns a 1x2 matrix: `[ok, value]`.

```
fn quebra() { return 1 / 0; }

r = pcall(quebra);
println(r[0]);   // 0 (failure)
println(r[1]);   // message + traceback

r = pcall(fn(a, b) { return a + b; }, 2, 3);
println(r[0]);   // 1
println(r[1]);   // 5
```

If the first argument is not a function, the result is `[0, "pcall: valor nao e uma funcao"]`. `pcall` is a special call form, not a value: do not write `g = pcall`. Unhandled errors show the line and frames in the REPL/runner.

8.8 Builtins as Functions

Builtins such as `println`, `len`, `str`, `abs`, and `sqrt` are called by name. They are not first-class values in v1.0; to pass an operation as an argument, wrap it in an anonymous function:

```
aplica(f, x) { return f(x); }
println(aplica(fn(v) { return v * v; }, 5));   // 25
```

Chapter 9

Strings

9.1 Basic Operations

```
s = "Ola, mundo";
println(len(s));       // 11
println(s[0]);         // O
println(s[-1]);        // o
println(upper(s));     // HELLO, WORLD
println(lower(s));     // hello, world
println(trim("  x  ")); // x
println(rep("ab", 3)); // ababab
println(reverse(s));   // reversed text
```

Strings are immutable: `s[0] = "X"` is an error. Negative indices count from the end. `len` counts characters (bytes in internal storage; for UTF-8 code points use `utf8_len`).

9.2 Slices

Slices work on strings and lists. The start is inclusive and the end is *exclusive*; bounds are clamped and do not raise an error.

```
s = "abcdef";
println(s[1:4]);   // bcd
println(s[:3]);    // abc
println(s[3:]);    // def
println(s[-2:]);   // ef
```

`sub(s, i[, j])` is different: it uses 0-based indices and the end is *inclusive*. See Appendix B.

9.3 Comparison and Concatenation

`+` concatenates two strings. `==` and `!=` compare content; `< > <= >=` are lexicographic between two strings. Comparing a string with a number is an error.

```
println("abc" + "def");   // abcdef
println("abc" == "abc");  // 1
println("abc" != "abd");  // 1
```

9.4 Conversion and Formatting

`str(x)` and `tostring(x)` convert to text. `format(fmt, ...)` returns a formatted string; `printf` prints it. The accepted conversions are `%s`, `%d`, `%i`, `%f`, `%g`, `%j`, `%c`, and `%%`. There is no width, precision, or flags: `%.2f` is an error.

```
nome = "Ana";
idade = 30;
msg = format("%s tem %d anos", nome, idade);
println(msg);               // Ana tem 30 anos
printf("[%s]\n", "ok");     // [ok]
```

9.5 Arduino String API

The functions below are free functions (not methods) and preserve string immutability; they all accept text and return text or a number depending on the operation.

```
s = "  Arduino  ";
println(trim(s));                    // Arduino
println(String(3.14159, 2));         // 3.14
println(String(255, 16));            // ff
println(charAt("abc", 1));           // b
println(indexOf("arduino", "du"));   // 2
println(toUpperCase("abc"));         // ABC

buf = [0, 0, 0, 0];
n = toCharArray("Az", buf);          // 2; buf = [65, 122, 0, 0]
println(buf[0]);                     // 65
```

Without a buffer, `getBytes`/`toCharArray` return a 1D list of bytes; with a buffer, they return the number of bytes copied. `getBytes` does not write a NUL; `toCharArray` writes a NUL if space remains. `String(float)` uses 2 decimal places; `String(int, base)` accepts base 2..36. There are also `compareTo`, `c_str`, `endsWith`, `equals`, `equalsIgnoreCase`, `lastIndexOf`, `length`, `remove`, `replace`, `reserve`, `setCharAt`, `startsWith`, `substring`, `toDouble`, `toFloat`, `toInt`, `toLowerCase`, and `toUpperCase`; `concat(s, ...)` concatenates text when the first argument is a string.

9.6 Patterns

The language implements a subset of Lua patterns. Indices are 0-based; a failed `match` returns `""` (empty string) for compatibility.

```
println(match("hello 123", "%d+"));        // 123
println(match("abc123", "(%a+)(%d+)"));    // abc (first capture)
println(gsub("a1b2", "%d", "#"));          // a#b#
println(gsub("abc", "(%a)", "[%1]"));      // [a][b][c]

lista = gmatch("a1 b22", "%d+");           // list ["1", "22"]
for (i = 0; i < len(lista); i += 1) {
    println(lista[i]);
}
```

Supported classes: `.` `%a` `%c` `%d` `%g` `%l` `%p` `%s` `%u` `%w` `%x` `%z` (uppercase inverts), sets `[...]` with ranges and `^`, quantifiers `*` `+` `-` `?`, anchors `^` `$`, captures `(...)`, `%bxy`, `%f[set]`, and backreferences `%1`..`%9`. `gsub` accepts textual replacement only.

9.7 UTF-8

```
s = "cafe";
println(utf8_len("caf\u00e9"));       // 4 code points (example)
println(utf8_codepoint("A", 0));      // 65
println(utf8_char(65));               // A
```

`utf8_len(s)` counts code points; `utf8_codepoint(s[, i])` returns the code point at byte `i` or `-1`; `utf8_char(cp)` encodes a valid code point. The source above uses `\u` only as an illustration; the v1.0 lexer does not interpret `\u`, so prefer literal UTF-8 strings.

Chapter 10

Lists and Matrices

10.1 1D Lists

A list is a 1xN matrix, indexed from 0.

```
v = [10, 20, 30];
println(v[0]);        // 10
println(v[-1]);       // 30
println(len(v));      // 3

v[1] = 99;
append(v, 40);        // mutates and returns the list
println(v);           // 1x4 matrix: 10, 99, 30, 40
```

`append` only works on 1D lists. `sort(v)` sorts numbers in ascending order (in place); `concat(v[, sep])` joins the elements into text.

10.2 Slices and Copies

Slices return a *copy* (mutating the copy does not change the original). The end is exclusive.

```
v = [0, 1, 2, 3, 4];
w = v[1:4];       // [1, 2, 3]
println(len(w));  // 3
w[0] = 99;
println(v[1]);    // 1 (original intact)
```

Assigning to a slice (`v[1:3] = ...`) is not supported.

10.3 2D Matrices

Lists of lists form matrices. `mat_index(M, i, j)` uses 1-based indices (the linear algebra convention), while direct indexing `M[i][j]` uses 0-based indices.

```
M = [[1, 2], [3, 4]];
println(M[0][1]);          // 2
println(mat_index(M, 1, 2)); // 2
println(rows(M));          // 2
println(cols(M));          // 2
```

Available operations: `det`, `trace`, `inv`, `transpose`, `dot`, `cross`, `matmul`, `identity`, `zeros`, `eigvals`, and `eigvecs`. The operators `+ - * /` and `@` (scalar) work on matrices.

Chapter 11

Tables

11.1 Creating and Accessing

Tables are key-value maps with reference semantics. The literal uses `key = value` pairs; literal keys are identifiers.

```
pessoa = {nome = "Ana", idade = 30};
println(pessoa.nome);          // Ana
println(pessoa["idade"]);      // 30
pessoa.idade += 1;
pessoa["cidade"] = "Recife";   // dynamic key
println(pessoa.cidade);        // Recife
```

Valid keys are text, number, or boolean. A missing key returns `nil`; assigning `nil` deletes the key.

```
println(pessoa.cargo);         // nil
println(has(pessoa, "cargo")); // 0
pessoa.cidade = nil;           // deletes
println(has(pessoa, "cidade"));// 0
```

11.2 Equality and Reference

Two tables are equal only if they are the same table (identity):

```
a = {x = 1};
b = a;
c = {x = 1};
println(a == b);   // 1
println(a == c);   // 0
```

11.3 Iteration

In v1.0, `pairs(t)` returns a list of pairs `[k, v]`; `ipairs(t)` returns the values of `t[0]`, `t[1]`, ... until the first missing key; `next(t[, k])` returns the next pair or an empty list at the end. The order is canonical/deterministic.

```
t = {a = 1, b = 2};
ps = pairs(t);
for (i = 0; i < len(ps); i += 1) {
    par = ps[i];
    println(par[0], "=", par[1]);
}
```

You can also iterate over sorted keys:

```
ks = keys(t);
for (i = 0; i < len(ks); i += 1) {
    println(ks[i], t[ks[i]]);
}
```

11.4 Textual Representation

`println(t)` shows `{key: value}` with sorted keys. Text keys appear in quotes; numbers and booleans do not.

11.5 Metatables

Tables and structs can have a metatable; see Chapter 13. Table operations also accept `rawget`, `rawset`, `has`, `remove`, `keys`, `len`, and `collectgarbage`.

Chapter 12

Structs and Classes

12.1 Structs

A struct groups fields and methods. The definition ends with `;`.

```
type Ponto {
    x;
    y;

    void mover(dx, dy) {
        x += dx;
        y += dy;
    }
};

p = Ponto{1, 2};
p.mover(3, 4);
println(p.x);   // 4
println(p.y);   // 6
println(p);     // Ponto(x=4, y=6)
```

Inside a method, fields are accessed directly by name (`x`, `y`), without `this.`. If you need a local variable with the same name as a field, choose another name.

The literal `Tipo{v1, v2, ...}` requires the exact field count, in definition order (root to leaf). You can also declare `Ponto p = {1, 2};`.

12.2 Classes and Inheritance

`class` is a struct with single inheritance, a constructor, and `super`.

```
class Conta {
    saldo;
    init(s) { saldo = s; }
    depositar(v) { saldo += v; return saldo; }
}

class Poupanca extends Conta {
    bonus;
    init(s, b) {
        super.init(s);
        bonus = b;
    }
    depositar(v) {
        saldo += v + bonus;
        return saldo;
    }
}
```

```
c = new Conta(100);
p = new Poupanca(100, 5);
println(c.depositar(10));   // 110
println(p.depositar(10));   // 115
```

`new T(args)` requires an `init` method (own or inherited). Fields are flattened in root→leaf order; methods resolve from child to parent. `super.m(args)` starts at the parent and uses the same object.

12.3 Reference and Equality

Instances are shared by reference: `b = a` points to the same object. The current implementation compares structs/classes *structurally* (type and fields); do not rely on identity to distinguish two instances with the same values. For explicit identity, add an `id` field and compare it.

12.4 Structs versus Tables

Use structs when the set of fields is fixed and part of the type; use tables for dynamic maps. Structs have methods and inheritance; tables have dynamic keys and more general metatables.

Chapter 13

Metatables and Garbage Collection

13.1 setmetatable and getmetatable

`setmetatable(t, mt)` associates a metatable with a table or struct; `nil` removes it. `getmetatable(t)` returns the metatable or `nil`.

13.2 Supported Metamethods

Metamethod	Effect
`__index`	lookup when the key/field is missing; may be a table or function
`__call`	makes the object callable; the object becomes the first argument
`__add`	covers `a + b`, `a + k`, `x = x + b`, and `x = x + k`
`__eq`, `__ne`	equality when both operands have the same type
`__tostring`	representation used by `println`, `str`, `printf %s`, `error`/`assert`
`__gc`	best-effort finalizer at script end or in `collectgarbage()`

Examples:

```
base = {saudacao = "ola"};
mt = {__index = base};
obj = setmetatable({nome = "Ana"}, mt);
println(obj.saudacao);        // hello

mtCall = {__call = fn(self, x) { return self.base + x; }};
soma = setmetatable({base = 10}, mtCall);
println(soma(5));             // 15

mtAdd = {__add = fn(a, b) { return a.x + b.x; }};
a = setmetatable({x = 2}, mtAdd);
b = setmetatable({x = 3}, mtAdd);
println(a + b);               // 5

mtStr = {__tostring = fn(self) { return "Pessoa(" + self.nome + ")"; }};
p = setmetatable({nome = "Ana"}, mtStr);
println(p);                   // Pessoa(Ana)
```

13.3 Cycle Collection

Tables, lists, structs, functions, and closure environments use reference counting and a cycle collector based on *trial deletion*. The collector runs at the end of each script/REPL entry and

on demand:

```
t = {};
t.self = t;
t = nil;
println(collectgarbage());   // >= 1

vivo = {};
vivo.self = vivo;
println(collectgarbage());   // 0 (still reachable)
```

The collector is not concurrent. Cycles kept alive by stacks of paused threads/coroutines are treated as roots and may not be collected. The bytecode of chunks loaded by `load`/`dofile` accumulates until the end of execution.

13.4 Limitations

There is still no `__newindex`, `__sub`, `__mul`, `__div`, `__mod`, `__pow`, `__unm`, `__lt`, `__le`, `__concat`, `__len`, `__pairs`, `__metatable`, or weak tables. Metatables work on tables and structs, not on lists/strings.

Chapter 14

Modules and Dynamic Code

14.1 include

`include("arquivo");` inserts the file contents at the directive point before compilation. The path is relative to the working directory; each file is included once per execution, which also breaks cycles. The directive requires parentheses and a string literal; the old `#` form is not accepted.

```
include("scripts/libs/showtext.xlb");
showtext("Ola");
```

14.2 .xlb Libraries

`.xlb` files are pure-language code libraries. `include` embeds the content; functions and constants become available in the script. Canonical example: `scripts/libs/showtext.xlb` and `scripts/script_26_biblioteca_xlb.isi`.

14.3 require

`require("arquivo")` loads an entire script as a module. The body runs once; the file's functions and variables become globals; the body's `return` value (or `0`) is returned and cached. `require` is resolved at compile time and is not a first-class value.

```
r = require("scripts/libs/util.xlb");
println(util_dobro(21));
r2 = require("scripts/libs/util.xlb");  // cached
```

A cycle `A -> B -> A` is a compilation error. A missing file or invalid syntax is also a compilation error (not catchable by `pcall`).

14.4 load and dofile

`load(codigo[, nome])` compiles a string as a chunk and returns a function. In turn, `dofile(caminho)` reads, compiles, and executes a file in isolation, returning the `return` value (or `0`).

```
f = load("return 1 + 2;");
println(f());                          // 3
r = dofile("tests/fixtures/l14_chunk.txt");
```

`eval(codigo)` is pure sugar for `load(codigo)()`: it compiles and runs the chunk immediately, returning the result. Both require `allow_modules` and `allow_fileio` in the sandbox. Loaded

chunks do not capture the caller's locals; only locals from the same lexical compilation scope. Do not use `load`/`dofile` concurrently with threads.

14.5 Native .mod Modules

`import("arquivo.mod")` loads a JSON manifest that points to a shared library (`.so`/`.dylib`). Native functions and variables enter the global namespace.

```
import("scripts/modules/motor_controller.mod");
println(MotorSpeedLimit);
MotorSpeedLimit = 5000;
r = Motor_Start(1500);
res = Motor_Read();
```

Accepted FFI types: signed and unsigned integers, `float32`, `float64`, `bool`, `string`, and `void`; at most 3 parameters. Variables with `readonly: true` cannot be written. See `docs/linguagem.md` for the complete manifest format and module lifecycle.

Chapter 15

Error Handling

15.1 Compilation and Runtime Errors

Compilation errors prevent execution and include the line. Runtime errors can be caught with `pcall`. `error(msg)` raises an error; `assert(cond[, msg])` raises if the condition is false.

```
fn valida(x) {
    assert(x > 0, "x must be positive");
    return x;
}

r = pcall(valida, -1);
println(r[0]);   // 0
println(r[1]);   // x must be positive (line ...)
```

15.2 Traceback

When an error is not caught, the output shows the message and frames:

```
ERRO EM TEMPO DE EXECUCAO: Divisao por zero (linha 7)
  em quebra (linha 10)
  em main (linha 14)
```

`pcall` returns the message already including the traceback in `r[1]`.

15.3 No try/catch

v1.0 has no `try/catch/finally` or `throw`. Use `pcall` to catch errors and `error` to raise them. For resource cleanup, close files explicitly or rely on automatic closing at the end of the script/REPL.

15.4 Sandbox

The embedding API can limit instructions and disable I/O and modules. In the runner/REPL, the REPL growth limits are configurable through the environment. `load/dofile/include/require/import(...)` respect the sandbox permissions.

Chapter 16

Threads and Coroutines

16.1 System Threads

A function declared with `thread` runs on a system thread. It does not receive parameters.

```
global n;
n = 0;

thread contador() {
    while (1) {
        n += 1;
        delay(100);   // milliseconds
    }
}

start contador();
delay(350);
stop contador();
println(n);           // approximately 3
```

`start`/`stop`/`pause`/`resume` control the thread. Globals are shared without a lock; avoid concurrent writes to the same variable. Graphics and input can only be used on the main thread. A successful script waits for threads to finish; on error, they are terminated.

16.2 Cooperative Coroutines

Coroutines use their own fixed-stack VM and are independent of system threads.

```
fn conta() {
    i = 0;
    while (i < 3) {
        coyield(i);
        i += 1;
    }
    return "fim";
}

co = cocreate(conta);
println(costatus(co));    // new

r = coresume(co);
println(r[0], r[1]);      // 1 0
r = coresume(co);
println(r[0], r[1]);      // 1 1
r = coresume(co);
println(r[0], r[1]);      // 1 2
r = coresume(co);
println(r[0], r[1]);      // 1 fim
```

```
println(costatus(co));    // dead
```

`coresume` returns `[ok, value]`. `coyield(v)` suspends and returns `v`; outside a coroutine it is an error. `costatus` returns `"novo"`, `"rodando"`, `"suspenso"`, or `"morto"`. Each coroutine uses a fixed stack; create few in embedded environments. There is no multi-value `coyield` and no `coyield` inside `pcall`.

Chapter 17

Standard Library

The library consists of global builtins. There are no `math.*`, `string.*`, `table.*`, etc. namespaces in v1.0. Appendix B provides a quick reference; the most used functions are below.

17.1 Console and Conversion

```
println("texto", 42, nil);     // text 42 nil
printf("%s=%d\n", "x", 7);     // x=7
nome = input("nome? ");        // reads a line from stdin
println(type(nome));           // string
println(tostring(42));         // 42
println(tonumber("ff", 16));   // 255
```

`println` accepts several arguments and prints them one by one, separated by spaces. `printf` does not add a newline; use `\n`.

17.2 Strings, Lists, and Tables

```
s = "  Ola  ";
println(upper(s));       // "  OLA  "
println(trim(s));        // "Ola"
println(sub("abcdef", 1, 3)); // bcd (inclusive end)
println(byte("A"));      // 65
println(char(65, 66));   // AB
println(format("%02d", 7));   // error: no flags/width

v = [3, 1, 2];
sort(v);                 // [1, 2, 3]
println(concat(v, "-")); // 1-2-3

t = {a = 1};
println(has(t, "a"));    // 1
println(keys(t));        // sorted list
```

17.3 Mathematics and Complex Numbers

```
println(abs(-3));        // 3
println(sqrt(9));        // 3
println(pow(2, 10));     // 1024
println(floor(3.7));     // 3
println(ceil(3.1));      // 4
println(round(3.5));     // 4
println(min(4, 2, 9));   // 2
println(max(4, 2, 9));   // 9
```

```
println(random());       // [0, 1)
println(random_int(1, 6));

z = 3 + 4j;
println(abs(z));         // 5
println(real(z));        // 3
println(imag(z));        // 4
println(conj(z));        // 3-4j
```

C99 math.h

In addition to the basic functions, the library exposes a C99 subset of `math.h` for `double` (complex numbers only where support already existed).

```
println(cbrt(27));          // 3
println(hypot(3, 4));       // 5
println(ldexp(0.5, 4));     // 8
println(fma(2, 3, 4));      // 10
a, b = frexp(8);            // 0.5 and 4
println(a);                 // 0.5
println(b);                 // 4
println(fpclassify(NAN));   // 0 (FP_NAN)
println(isfinite(INF));     // 0
println(M_PI);              // ~3.14159
```

The list includes `acos`, `asin`, `atan`, `acosh`, `asinh`, `atanh`, `cosh`, `sinh`, `tanh`, `exp2`, `expm1`, `log2`, `log1p`, `cbrt`, `erf`, `erfc`, `tgamma`, `lgamma`, `trunc`, `rint`, `nearbyint`, `fabs`, `hypot`, `fdim`, `fmax`, `fmin`, `copysign`, `nextafter`, `remainder`, `lround`, `llround`, `lrint`, `llrint`, `ilogb`, `logb`, `ldexp`, `scalbn`, `scalbln`, `fma`, `nan`, `fpclassify`, `isfinite`, `isinf`, `isnan`, `isnormal`, `signbit`, `isgreater`, `isgreaterequal`, `isless`, `islessequal`, `islessgreater`, and `isunordered`. `frexp`, `modf`, and `remquo` return multiple values.

Constants are literals resolved at compile time and cannot be assigned: `M_E`, `M_LOG2E`, `M_LOG10E`, `M_LN2`, `M_LN10`, `M_PI`, `M_PI_2`, `M_PI_4`, `M_1_PI`, `M_2_PI`, `M_2_SQRTPI`, `M_SQRT2`, `M_SQRT1_2`, `M_PHI`; aliases `PI`, `E`, `TAU`, `PHI`, `DEG_TO_RAD`, `RAD_TO_DEG`, `INF`, `NAN`; limits `HUGE_VAL`, `INFINITY`, `MAXFLOAT`, `FLT_MAX`, `DBL_MAX`, `FLT_EPSILON`, `DBL_EPSILON`; classification `FP_NAN` (0), `FP_INFINITE` (1), `FP_ZERO` (2), `FP_SUBNORMAL` (3), `FP_NORMAL` (4); and `MATH_ERRNO` (1), `MATH_ERREXCEPT` (2), `math_errhandling` (3).

17.4 Time

```
println(time());        // seconds since the epoch
println(clock());       // milliseconds since startup
println(os_date());     // formatted local date/time
```

17.5 Files

Handles are monotonic integers; a failing `fopen` returns `0` (the `NULL` constant). Argument order follows C: data first, file last, except `fprintf(f, fmt, ...)`.

```
f = fopen("dados.txt", "w");
```

```
if (f == NULL) {
    println("nao abriu");
} else {
    fputs("linha 1\n", f);
    fprintf(f, "%s=%d\n", "x", 42);
    fclose(f);
}

r = fopen("dados.txt", "r");
println(fread(100, r));
println(fgets(r));
fclose(r);
```

17.6 Sound

```
sound_backend("device");
h = sound_load("a.wav");
sound_play(h);
sound_set_volume(h, 0.5);
delay(1000);
sound_stop(h);
sound_unload(h);
```

Real audio output is optional: use `sound_backend("device")` in the script or `MYINTERP_SOUND=device`. Without that (or without raylib), the backend is software and deterministic. `sound_backend("software")` returns to silent mode; `MYINTERP_SOUND=software` also forces software.

Chapter 18

Graphics

The model is inspired by Processing. The `software` backend is headless and writes `.ppm` images; the `raylib` backend opens a real window when compiled with support.

18.1 setup and draw

If the script defines `setup()`, it runs before any other statement. Use `setDraw(fn)` (or `onDraw`) to register the frame callback.

```
void setup() {
    size(320, 200);
    setDraw(draw);
}

void draw() {
    background(20, 20, 40);
    fill(255, 160, 0);
    circle(mouseX(), mouseY(), 20);
    text("q para sair", 5, 5);
    if (keyPressed() && key() == "q") { quit(); }
}
```

Without `draw`/`setDraw`, use the manual loop:

```
size(80, 60);
i = 0;
while (i < 60 && show()) {
    background(0);
    fill(255, 200, 0);
    circle(10 + i, 30, 12);
    i += 1;
}
```

18.2 Drawing

```
size(80, 60);
background(20, 20, 40);
fill(255, 128, 0);
stroke(255, 255, 255);
strokeWeight(1);
noFill(); noStroke();
rect(5, 5, 30, 20);       // top-left corner
circle(60, 15, 20);       // center, diameter
ellipse(40, 45, 30, 14);  // center, axes
line(0, 59, 79, 0);
point(79, 59);
```

```
textSize(1);
font("8x8");              // "8x8" (default) or "5x7"
fill(255);
text("Isiliel", 5, 40);   // "\n" breaks the line
show();
save_screen("out.ppm");   // .ppm; .png with raylib
println(width(), height());
println(frameCount());
```

18.3 Input and Images

```
println(mouseX(), mouseY());
println(mousePressed());
println(keyPressed());
println(key());

img = loadImage("foto.ppm");
image(img, 10, 10);
image(img, 10, 10, 64, 40);
println(imageWidth(img), imageHeight(img));
unloadImage(img);
```

Multiple keys in the same frame are coalesced: `key()` returns only the last one. ESC and the X button do not close the window; use `quit()` or Ctrl+C. Drawing without `size()`, or calling `backend()` after `size()`, raises an error with the line number.

Chapter 19

Embedding the Interpreter in C++

The public API is in `interp.h`. Minimal example:

```
#include "interp.h"

int main() {
    InterpOptions opts;
    opts.capture = true;          // stdout -> result.output
    opts.step_limit = 100000;     // instruction budget
    opts.allow_fileio = false;    // sandbox
    opts.allow_modules = false;

    InterpResult r = interp_run("println(6 * 7);", opts);
    if (!r.ok) {
        // r.error contains the message
        return 1;
    }
    // r.output contains "42\n"
    return 0;
}
```

19.1 Options

Field	Meaning
`verbose`	prints execution banners
`capture`	redirects stdout to `InterpResult.output`
`step_limit`	maximum VM instructions (0 = no limit)
`allow_fileio`	enables `fopen`, `include`, `require`
`allow_modules`	enables `import(...)` and native modules

19.2 API Functions

- `interp_run(source, opts)`: compiles and runs; always restores threads, files, screen, modules, and sandbox on exit.
- `interp_register_fn(nome, arity, handler)`: registers a host function as a builtin. Call before running.
- `interp_set_output(std::ostream* os)`: redirects output; `nullptr` restores `std::cout`.
- `interp_reset()`: clears symbols, bytecode, threads, files, screen, and modules for the next single-state run.
- `run_pipeline`, `run_file`, `disasm_file`: low-level variants used by the CLI.

19.3 Builtin Signature

A registered function receives an array of `Value`, the argument count, and a pointer to the result. See `vm.h` for `BuiltinHandler` and `defs.h` for `Value`. Errors should be thrown as `std::runtime_error`; the VM appends the line.

Chapter 20

Tools and Debugging

20.1 Disassembler

```
./build/isirun --disasm script.isi
```

Shows the bytecode with opcode names and operands. Useful for understanding optimizations and debugging the VM.

20.2 Tests and Sanitizers

```
make test
make asan
make fuzz FUZZ_ITERS=2000
make bench
```

The suite covers scripts, expected errors, the REPL, the disassembler, and the API. The fuzzer performs deterministic mutation of scripts; ASan/UBSan check memory and undefined behavior. The benchmark compares against Lua 5.4 and writes results to `bench/comparacao.html`.

20.3 Project Structure

File	Role
`lexer.cpp/h`	lexical analysis
`compiler.cpp/h`	parser and bytecode generation
`vm.cpp/h`	stack VM and builtins
`symbols.cpp/h`	symbol table and scopes
`interp.cpp/h`	pipeline and embedding API
`run.cpp`	CLI and REPL
`graphics/`	graphics HAL and backends
`scripts/`	runnable examples
`tests/`	automated suite

20.4 Debugging in VS Code

The project includes an `F5` configuration to build and debug a script. The debug build uses `-O0 -g3`; for performance use `make release`.

Chapter 21

Known Limitations and Differences

This section summarizes what does *not* exist or differs from the project documentation in v1.0.

21.1 Language

- `true`/`false` are 1/0; `type(true)` returns `number`. The internal `VAL_BOOL` type exists, but it is not produced by literals/comparisons in v1.0.
- `&&`/`||` do not short-circuit and return 1/0. `!` and `~` are bitwise negation; there is no logical negation.
- There is no `local`, `let`, or `const`; assignment inside a function is implicitly local.
- Only C-style `for` exists. There is no `for k,v in ...` or native iterators; `pairs`/`ipairs` return lists.
- There is no `and`/`or`/`not`, `#`, `..`, or `//` operator. Use `floordiv` and `+`.
- There is no IIFE or postfix call/indexing on arbitrary expressions: `f()[0]` and `fn(x){...}(1)` are not supported. Use temporaries.
- Named functions and methods do not capture outer locals; only anonymous functions do. There is no forward capture.
- Table literals accept only identifier keys; use `t[chave] = v` for dynamic keys.
- There are no library namespaces (`math.*`, `string.*`, `table.*`) or first-class builtins.
- There is no `try`/`catch`/`finally`/`throw`. Use `pcall`.

21.2 Metatables and Library

- Missing: `__newindex`, `__sub`, `__mul`, `__div`, `__mod`, `__pow`, `__unm`, `__lt`, `__le`, `__concat`, `__len`, `__pairs`, `__metatable`, and weak tables.
- Metatables do not work on lists or strings.
- `match` returns `""` on failure (not `nil`); `gmatch` returns a list (not an iterator); `gsub` only accepts textual replacement.
- `printf`/`format` do not accept width, precision, or flags.
- `require` is resolved at compile time and is not a first-class value; there is no `package.path` or `package.loaded`.
- There is no `debug.*` or stack introspection.

21.3 Concurrency and Memory

- The cycle collector is not concurrent; cycles in paused thread/coroutine stacks may be retained.

- `load`/`dofile` bytecode accumulates until the end of execution; there is no per-chunk release.
- Threads share globals without a lock; concurrent writes are the user's responsibility.
- Coroutines have a fixed stack and single-value `coyield`; there is no `coyield` inside `pcall`.

21.4 Roadmap

The only structural item still pending in the documents is E18 (register VM), which is a performance optimization. The functional gaps above are candidates for future phases: real booleans, short-circuiting, block scope, named closures, postfix/IIFE, `for-in`, namespaces, complete metatables, and error handling with `try/catch`.

Chapter A

Grammar Summary

The complete grammar is in `docs/gramatica.ebnf`. Summary:

```
program        = { statement } EOF ;
block          = "{", { statement }, "}" ;

statement      = block | if_statement | while_statement
               | for_statement | repeat_statement
               | "return", [ expression_list ], ";"
               | "break", ";" | "continue", ";"
               | "global", identifier, { ",", identifier }, ";"
               | "goto", identifier, ";"
               | "::", identifier, "::"
               | type_definition | class_definition
               | thread_definition | thread_command
               | include_directive | import_directive
               | function_definition | scalar_declaration
               | expression_statement ;

if_statement   = "if", "(", expression, ")", statement,
                 { "elseif", "(", expression, ")", statement },
                 [ "else", statement ] ;

while_statement = "while", "(", expression, ")", statement ;

for_statement  = "for", "(", [ expression ], ";",
                 [ expression ], ";",
                 [ expression ], ")", statement ;

repeat_statement = "repeat", statement, "until",
                   [ "(" ], expression, [ ")" ], [ ";" ] ;

function_definition = [ type_prefix ], [ "fn" ], identifier,
                      "(", [ parameter_list ], ")", block ;

anonymous_function = [ "fn" ], "(", [ parameter_list ], ")", block ;

type_definition  = "type", identifier, "{", { struct_member }, "}", ";" ;
class_definition = "class", identifier, [ "extends", identifier ],
                   "{", { struct_member }, "}", [ ";" ] ;
```

Precedence (lowest to highest): assignment, `||`, `&&`, `|`, `^`, `&`, `== !=`, `< > <= >=`, `« »`, `+ -`, `* / %`, unary, postfix.

Chapter B

Quick Builtin Reference

This list covers the most used functions. For the complete list and arity details, see `docs/linguagem.md` and the code in `vm.cpp`, `patterns.cpp`, `fileio.cpp`, `threads.cpp`, `sound.cpp`, and `graphics/`.

Function	Summary
`println(...)`	prints values with a newline
`printf(fmt, ...)`	prints formatted output, no newline
`format(fmt, ...)`	returns a formatted string
`input([prompt])`	reads a line from stdin
`type(x)`	type name
`str(x)`, `tostring(x)`	textual representation
`tonumber(x[, base])`	converts to a number or `nil`
`assert(cond[, msg])`	error if the condition is false
`error(msg)`	raises an error
`collectgarbage()`	runs the cycle collector
`len(x)`	length of string/list/table
`append(lista, x)`	appends to the end of a 1D list
`sort(v)`	sorts numbers (in place)
`concat(v[, sep])`	joins elements into text
`upper(s)`, `lower(s)`	uppercase/lowercase
`trim(s)`	removes leading/trailing spaces/tabs
`rep(s, n)`	repeats a string
`reverse(s)`	reverses a string
`find(s, sub)`	0-based index or -1
`sub(s, i[, j])`	0-based substring, inclusive end
`byte(s[, i])`	byte code
`char(b1, ...)`	string from bytes
`match(s, pat[, init])`	first match/capture
`gmatch(s, pat)`	list of matches
`gsub(s, pat, repl)`	textual substitution
`utf8_len(s)`	code points
`utf8_codepoint(s[, i])`	code point or -1
`utf8_char(cp)`	encodes a code point
`String(...)`	text constructor (float: 2 decimals; int: base)
`charAt`, `compareTo`, `c_str`	Arduino String API
`endsWith`, `equals`, `equalsIgnoreCase`	text comparisons
`getBytes`, `toCharArray`	byte list or buffer
`indexOf`, `lastIndexOf`, `length`	search and length
`remove(s,i[,n])`, `replace`, `setCharAt`	immutable editing
`reserve`, `startsWith`, `substring`	Arduino utilities

Function	Summary
`toDouble`, `toFloat`, `toInt`	text conversion
`toLowerCase`, `toUpperCase`	upper/lower case
`has(t, k)`	1 if the key exists
`remove(t, k)`	removes a key; 1 if it existed
`keys(t)`	sorted list of keys
`rawget(t, k)`, `rawset(t, k, v)`	access without metamethods
`next(t[, k])`	next pair or empty list
`pairs(t)`	list of pairs
`ipairs(t)`	values of `t[0..]`
`select(n, ...)`	selects arguments
`unpack(v[, i[, j]])`	returns elements
`abs`, `sqrt`, `pow`	mathematical functions
`sin`, `cos`, `tan`	trigonometry (complex numbers)
`atan2(y, x)`	arc tangent
`log`, `log10`, `exp`	logarithms/exponential
`floor`, `ceil`, `round`	rounding
`floordiv(a, b)`	integer division
`fmod(a, b)`	remainder with dividend sign
`min(...)`, `max(...)`	minimum/maximum
`random()`, `random_int(a,b)`	random numbers
`conj`, `real`, `imag`	complex numbers
`rect(re,im)`, `polar(m,ang)`	complex construction
`acos`, `asin`, `atan`	inverse trigonometry
`cosh`, `sinh`, `tanh`	hyperbolic trigonometry
`acosh`, `asinh`, `atanh`	inverse hyperbolic
`cbrt`, `erf`, `erfc`	cube root and error
`tgamma`, `lgamma`	gamma functions
`exp2`, `expm1`, `log2`, `log1p`	extra exp/log
`trunc`, `rint`, `nearbyint`	C99 rounding
`hypot`, `fdim`, `fmax`, `fmin`	C99 binary
`copysign`, `nextafter`, `remainder`	C99 binary
`lround`, `llround`, `lrint`, `llrint`	integer rounding
`ilogb`, `logb`, `ldexp`, `scalbn`, `scalbln`	exponent/scale
`fma`, `nan`	C99
`frexp`, `modf`, `remquo`	multiple return
`fpclassify`, `isnormal`, `signbit`	classification
`isgreater`, `isless`, `isunordered`	C99 comparison
`PI`, `E`, `TAU`, `M_PI`, `M_*`	mathematical constants
`FP_*`, `math_errhandling`	classification/environment
`time()`, `clock()`	clocks
`os_time()`, `os_clock()`	aliases

Function	Summary
`os_date([fmt])`	local date/time
`mat_index(M,i,j)`	1-based linear index
`rows(M)`, `cols(M)`	dimensions
`identity(n)`, `zeros(n)`	special matrices
`det`, `trace`, `inv`, `transpose`	linear algebra
`dot`, `cross`, `matmul`	products
`eigvals(M)`, `eigvecs(M)`	eigenvalues/eigenvectors
`fopen`, `fclose`	open/close file
`fread`, `fwrite`, `fputs`	read/write
`fgets`, `fgetc`, `fputc`	line/character
`fseek`, `ftell`, `rewind`	position
`feof`, `ferror`	state
`fprintf(f, fmt, ...)`	formatted write
`cocreate(f)`	creates a coroutine
`coresume(co, ...)`	resumes a coroutine
`coyield(v)`	suspends a coroutine
`costatus(co)`	coroutine state
`start/stop/pause/resume f()`	controls a thread
`delay(ms)`	sleeps for milliseconds
`load(src[, nome])`	compiles a chunk
`dofile(path)`	executes a file
`require("path")`	loads a module (compile time)
`setmetatable(t, mt)`	sets a metatable
`getmetatable(t)`	gets a metatable
`rawget`, `rawset`	direct access
`size(w,h)`, `background(...)`	graphics
`fill(...)`, `stroke(...)`	colors
`rect`, `circle`, `ellipse`, `line`	primitives
`text(...)`, `font(...)`, `textSize(...)`	text
`show()`, `quit()`	frame/termination
`loadImage`, `image`, `unloadImage`	images
`mouseX`, `mouseY`, `keyPressed`, `key`	input
`sound_load`, `sound_play`, `sound_stop`	sound

Chapter C

Complete Programs

C.1 FizzBuzz

```
for (i = 1; i <= 30; i += 1) {
    if (i % 15 == 0) {
        println("FizzBuzz");
    } elseif (i % 3 == 0) {
        println("Fizz");
    } elseif (i % 5 == 0) {
        println("Buzz");
    } else {
        println(i);
    }
}
```

C.2 Iterative and Recursive Fibonacci

```
fib_iter(n) {
    a = 0; b = 1;
    for (i = 0; i < n; i += 1) {
        t = a + b;
        a = b;
        b = t;
    }
    return a;
}

fib_rec(n) {
    if (n < 2) { return n; }
    return fib_rec(n - 1) + fib_rec(n - 2);
}

println(fib_iter(20));   // 6765
println(fib_rec(10));    // 55
```

C.3 Counter Closure

```
contador() {
    n = 0;
    return fn() {
        n += 1;
        return n;
    };
}
```

```
c = contador();
println(c());   // 1
println(c());   // 2
println(c());   // 3
```

C.4 Table and Iteration

```
estoque = {
    maca = 10,
    banana = 5,
    laranja = 8
};

ks = keys(estoque);
for (i = 0; i < len(ks); i += 1) {
    k = ks[i];
    println(k, "=", estoque[k]);
}

ps = pairs(estoque);
for (i = 0; i < len(ps); i += 1) {
    println(ps[i][0], ps[i][1]);
}
```

C.5 Class with Inheritance

```
class Animal {
    nome;
    init(n) { nome = n; }
    falar() { return "..."; }
}

class Cachorro extends Animal {
    init(n) { super.init(n); }
    falar() { return "au au"; }
}

a = new Animal("bicho");
d = new Cachorro("rex");
println(a.falar());   // ...
println(d.falar());   // au au
println(d.nome);      // rex
```

C.6 Metatable for Addition

```
mt = {__add = fn(a, b) { return a.x + b.x; }};
a = setmetatable({x = 2}, mt);
b = setmetatable({x = 3}, mt);
println(a + b);   // 5
```

C.7 Coroutine

```
fn produtor() {
    for (i = 1; i <= 3; i += 1) {
        coyield(i * 10);
    }
    return "fim";
}

co = cocreate(produtor);
while (1) {
    r = coresume(co);
    println(r[1]);
    if (r[1] == "fim") { break; }
}
```

C.8 Simple Graphics

```
void setup() {
    size(160, 120);
    setDraw(quadro);
}

void quadro() {
    background(15, 15, 35);
    fill(255, 180, 0);
    circle(80, 60, 40);
    if (keyPressed() && key() == "q") { quit(); }
}
```

Chapter D

Glossary

Array	0-based list/matrix, represented by `VAL_ARRAY`.
Builtin	Native function registered in the VM and called by name.
Bytecode	Instructions produced by the compiler and executed by the VM.
Closure	Anonymous function that captures local variables from the enclosing scope.
Coroutine	Cooperative execution that can be suspended and resumed.
Metatable	Table that defines metamethods for another table/struct.
Metamethod	Special field such as `__index` or `__add`.
REPL	Interactive read-eval-print environment.
Sandbox	Restrictions on I/O, modules, and instruction count.
Struct	Type with fields and methods, defined by `type`.
Table	Key-value map with reference semantics.
Stack VM	Virtual machine that uses a stack of values.